5 Grafos dirigidos

Formalmente, um ***grafo dirigido*** é um par (V, E) onde V é um conjunto finito (vértices) e $E \subseteq V \times V$. Com a restrição $(v, v) \notin E$ para todo $v \in V$, chamamos ***grafo dirigido simples*** (podem existir (u, v) e (v, u) simultaneamente).

Dizemos que a aresta, ou ***arco***, $(v, u) \in E$ ***sai de*** v e ***chega em*** u. Também, definimos os subconjuntos $N^+(v), N^-(v) \subseteq V$, de *vizinhos* de cada $v \in V$

$$N^+(v) \coloneqq \{u \in V \, (v, u) \in E\} \quad \text{e} \quad N^-(v) \coloneqq \{u \in V \, (u, v) \in E\}.$$

Definimos, para cada $v \in V$, o ***grau de saída***, $d^+(v)$, e o ***grau de entrada***, $d^-(v)$, respectivamente, por

$$d^+(v) \coloneqq |N^+(v)| \quad \text{e} \quad d^-(v) \coloneqq |N^-(v)|$$

e é fácil de verificar que vale

$$\sum_{v \in V} d^+(v) = \sum_{v \in V} d^-(v) = |E|,$$

pois cada arco (u, v) é contado uma vez em $d^+(u)$ e uma vez em $d^-(v)$.

O ***grafo subjacente*** ao grafo dirigido $G = (V, E)$ é o grafo SG dado por

$$SG = \left(V, \bigcup_{\substack{(u,v) \in E \\ u \neq v}} \{\{u, v\}\}\right)$$

5.1 Representação computacional e percurso

De modo análogo ao caso não dirigido, assumiremos que nos problemas computacionais os vértices de G são $\{1, 2, \ldots, n\}$ (caso contrário construímos um isomorfismo).

Como antes, um grafo dirigido (V, E) pode ser representado por uma matriz A quadrada $|V| \times |V|$ definida por

$$A(i, j) = \begin{cases} 1, & \text{se } (i, j) \in E \\ 0, & \text{caso contrário,} \end{cases}$$

ou seja, é uma matriz com entradas 0 e 1 dada por $a_{i,j} = 1$ se e só se (i, j) é arco. Notemos que a matriz agora pode não ser simétrica. Para o grafo G representado na Figura 5.1, a matriz de adjacência é

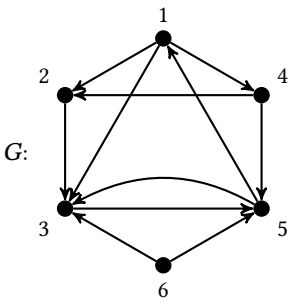


Figura 5.1: Grafo dirigido representado na matriz (5.2) e na lista de adjacências da Figura 5.2.

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \end{bmatrix}. \tag{5.2}$$

Também podemos representar um grafo por sua ***lista de adjacência dirigida***, dada por um vetor $N^+[\,]$ de listas ligadas. A lista $N^+[i]$ contém os vizinhos $N^+(i)$ do vértice i e cada nó dessa lista é composto por uma variável que armazena um vértice e um ponteiro que aponta para o próximo nó da lista.

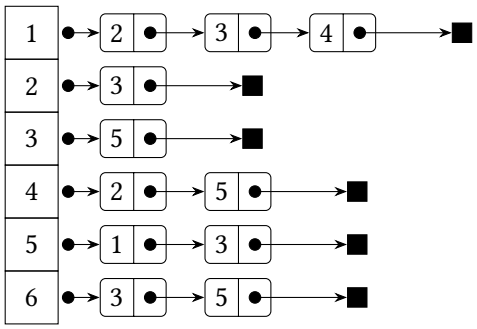


Figura 5.2: Lista de adjacência de um grafo dirigido com 6 vértices.

5.1.1 Busca em profundidade

Uma busca em profundidade rotulada, algoritmo 3 na página 32 reproduzido parcialmente na margem ao lado, executada num grafo dirigido resulta numa classificação das arcos em quatro tipos, ao invés dos dois tipos (árvore/retorno) que ocorrem no caso não dirigido.

```
1 cont = 0
2 cont ← cont + 1
3 chega[w] ← cont
4 para cada u ∈ N⁺(w) faça
5     se chega[u] = 0 então
6         p[u] ← w
7         BP(G, u)
8 cont ← cont + 1
9 sai[w] ← cont.
```

Considere uma execução completa da busca em profundidade rotulada, que recomeça em um vértice ainda não visitado até visitar todos. Seja $\mathtt{p}[v]$ o pai de v, com $\mathtt{p}[v] = \mathtt{nil}$ se v é raiz de uma árvore da busca, e defina $\mathtt{p}^{(0)}[v] = v$ e $\mathtt{p}^{(t)}[v] = \mathtt{p}[\mathtt{p}^{(t-1)}[v]]$ enquanto o resultado está definido. Dizemos que u é ***ancestral próprio*** de v (e v ***descendente próprio*** de u) se $\mathtt{p}^{(t)}[v] = u$ para algum $t \geq 1$. Se $\mathtt{p}[u] = v$, dizemos que u é ***filho*** de v e que v é ***pai*** de u.

Proposição 5.1: Classificação das arestas

Após uma busca em profundidade rotulada, todo arco $(u, v) \in E$ satisfaz exatamente uma das condições

1. *arco pai–filho*: $\mathtt{p}[v] = u$;
2. *retorno ascendente*: v é ancestral próprio de u, caso em que $[\mathtt{chega}[u], \mathtt{sai}[u]] \subset [\mathtt{chega}[v], \mathtt{sai}[v]]$;
3. *retorno descendente*: v é descendente próprio de u e $\mathtt{p}[v] \neq u$, caso em que $[\mathtt{chega}[v], \mathtt{sai}[v]] \subset [\mathtt{chega}[u], \mathtt{sai}[u]]$;

4. *arco transversal*: chega[v] < sai[v] < chega[u] < sai[u].

(A prova usa que, para $x \neq y$, os intervalos [chega, sai] são disjuntos ou um contém o outro, e que chega[u] < chega[v] implica v descendente de u se v é branco quando u é explorado; veja Cormen et al. (2012).)

A figura abaixo esquematiza uma busca em profundidade rotulada no grafo dirigido do exemplo 5.2.

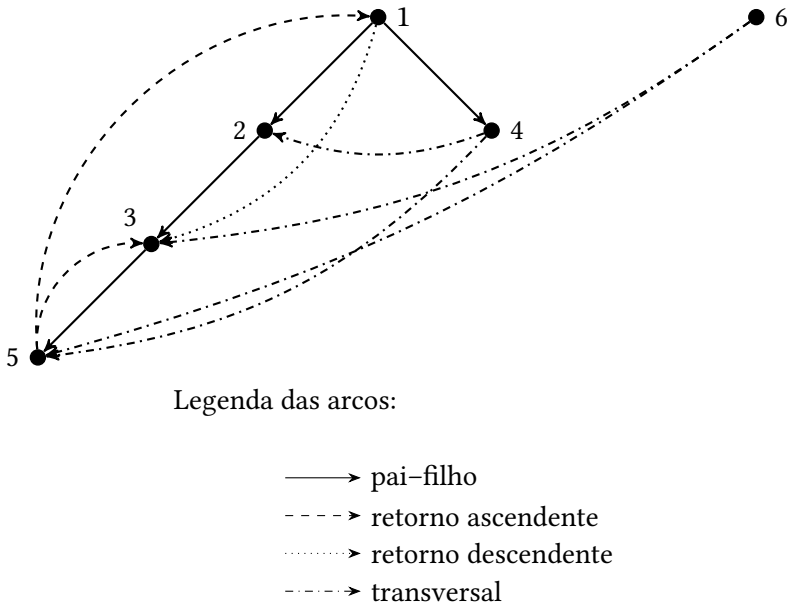


Figura 5.3: Busca em profundidade rotulada.

Exercício 83. A Figura 5.5 é o resultado de uma busca em profundidade rotulada no grafo dirigido da Figura 5.4.

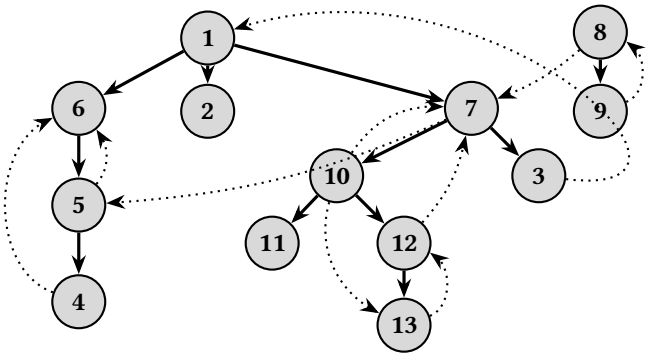


Figura 5.5: Esquema de uma busca rotula em 5.4

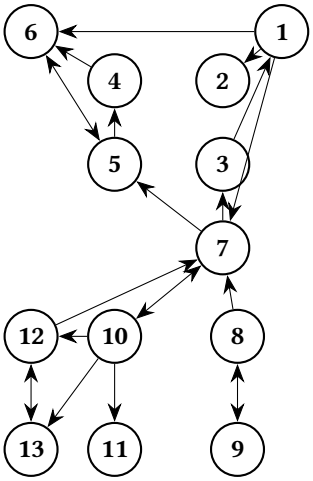


Figura 5.4: Exemplo de BP em grafo dirigido

Classifique as arestas do grafo dirigido após a busca em profundidade.

5.2 Grafos dirigidos acíclicos e ordenação topológica

Em muitos problemas os arcos de um grafo dirigido representam relações de precedência: um arco (u, v) indica que u deve vir antes de v. É o caso de tarefas de um projeto em que algumas só podem começar depois que outras terminam, de disciplinas com pré-requisitos, ou de arquivos de um programa em que a compilação de um depende de outro. Nessas situações queremos uma sequência que respeite todas as precedências. Isso é impossível se as precedências formam um circuito: u antes de v, v antes de w e w antes de u. Nesta seção mostramos que essa é a única obstrução e como encontrar uma tal sequência em tempo linear.

Nesta seção todos os grafos dirigidos são simples, isto é, sem laços.

Definição 5.1: Caminho e circuito orientados

Um ***caminho orientado*** em G é uma sequência de vértices distintos $x_1, \ldots, x_k$, com $k \geq 1$ e $(x_i, x_{i+1}) \in E(G)$ para todo $i \in \{1, \ldots, k-1\}$.
Um ***circuito orientado*** é uma sequência $x_1, \ldots, x_k, x_1$, em que $x_1, \ldots, x_k$ é um caminho dirigido com $k \geq 2$ e $(x_k, x_1) \in E(G)$. (Como não há laços, $k = 1$ não ocorre; para $k = 2$ o circuito usa os arcos (x_1, x_2) e (x_2, x_1).)

Definição 5.2: grafo dirigido acíclico

Um ***grafo dirigido acíclico*** (GDA, em inglês *DAG*) é um grafo dirigido que não contém circuito orientado.

Exemplo 5.1

Considere cinco disciplinas a, b, c, d, e com os seguintes pré-requisitos: a é pré-requisito de b e de c; b e c são pré-requisitos de d; c é pré-requisito de e. O grafo dirigido G dado por $V(G) = \{a, b, c, d, e\}$ e

$$E(G) = \{(a, b), (a, c), (b, d), (c, d), (c, e)\}$$

modela essa situação e é acíclico. A sequência a, b, c, d, e respeita os pré-requisitos, assim como a, c, e, b, d. Já a sequência a, b, d, c, e não serve, pois c é pré-requisito de d mas vem depois dele.

Definição 5.3: ordenação topológica

Seja G um grafo dirigido com n vértices. Uma ***ordenação topológica*** de G é uma ordenação $v_1, v_2, \ldots, v_n$ dos vértices de G, cada vértice aparecendo exatamente uma vez, tal que

$$(v_i, v_j) \in E(G) \implies i < j.$$

Escrevendo $u \prec v$ quando u aparece antes de v na ordenação, a condição é que $u \prec v$ para todo arco (u, v) de G. O ***problema da ordenação topológica*** é: dado um grafo dirigido G, determinar uma ordenação topológica de G ou concluir que não existe.

Se dispomos os vértices em linha, na ordem da ordenação, todos os arcos apontam para a direita, como no segundo diagrama da Figura 5.6. As ordenações do Exemplo 5.1 são exatamente as sequências de disciplinas que respeitam os pré-requisitos.

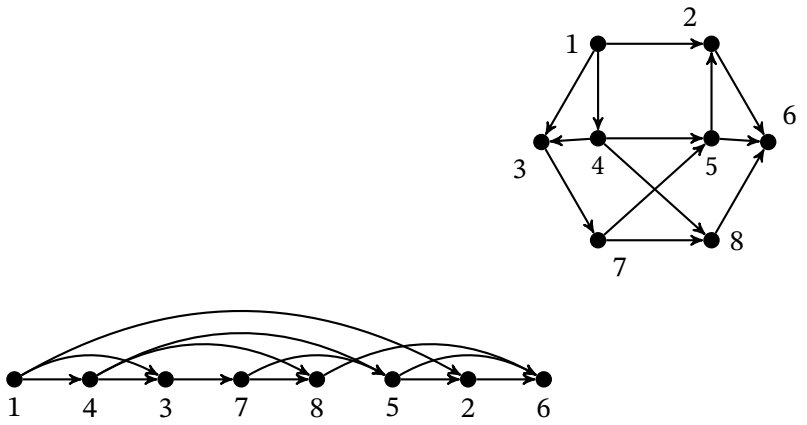


Figura 5.6: Um grafo dirigido e uma ordenação topológica de seus vértices: $1 \prec 4 \prec 3 \prec 7 \prec 8 \prec 5 \prec 2 \prec 6$.

Dizemos que um vértice v é uma ***fonte*** se $d^-(v) = 0$, isto é, nenhum arco chega em v, e que v é um ***sumidouro*** se $d^+(v) = 0$, isto é, nenhum arco sai de v. Em toda ordenação topológica o primeiro vértice é uma fonte e o último é um sumidouro: um arco que chegasse no primeiro, ou saísse do último, teria o outro extremo antes do primeiro ou depois do último vértice. Em um GDA sempre existem uma fonte e um sumidouro, e isso é o ponto de partida para provar a existência de ordenações topológicas.

Proposição 5.2: Fontes e sumidouros

Todo grafo dirigido acíclico com pelo menos um vértice tem uma fonte e um sumidouro.

Demonstração. Seja G um GDA com pelo menos um vértice. Um *caminho orientado* em G é uma sequência de vértices distintos $x_1, \ldots, x_k$ com $(x_i, x_{i+1}) \in E(G)$ para todo $i \in \{1, \ldots, k-1\}$ (Seção 5.3). Uma sequência com um só vértice é um caminho orientado e todo caminho

orientado tem no máximo $|V(G)|$ vértices, logo existe um caminho orientado $P = x_1, x_2, \dots, x_k$ com $k \geq 1$ máximo.

Suponha que existe um arco $(u, x_1) \in E(G)$. Como G não tem laços, $u \neq x_1$. Se $u \notin \{x_1, \dots, x_k\}$ então $u, x_1, \dots, x_k$ é um caminho orientado com $k+1$ vértices, contradizendo a maximalidade de P. Se $u = x_i$ para algum $i \in \{2, \dots, k\}$ então $x_1, \dots, x_i, x_1$ é um circuito orientado em G, contradizendo que G é acíclico. Logo nenhum arco chega em x_1, isto é, $d^-(x_1) = 0$ e x_1 é uma fonte.

Analogamente, suponha que existe um arco $(x_k, w) \in E(G)$; então $w \neq x_k$, pois não há laços. Se $w \notin \{x_1, \dots, x_k\}$, o caminho $x_1, \dots, x_k, w$ contradiz a maximalidade de P; se $w = x_i$ com $i \in \{1, \dots, k-1\}$, então $x_i, \dots, x_k, x_i$ é um circuito dirigido em G, uma contradição. Portanto $d^+(x_k) = 0$ e x_k é um sumidouro. □

Teorema 5.1: Existência de ordenação topológica

Um grafo dirigido G admite uma ordenação topológica se e somente se G é acíclico.

Demonstração. ($\Rightarrow$) Seja $v_1, \dots, v_n$ uma ordenação topológica de G e denote por $\mathrm{pos}(v)$ o índice de v nessa ordenação. Suponha, por absurdo, que $C = x_1, \dots, x_k, x_1$ é um circuito orientado em G. Os arcos (x_i, x_{i+1}) dão

$$\mathrm{pos}(x_1) < \mathrm{pos}(x_2) < \cdots < \mathrm{pos}(x_k),$$

e o arco (x_k, x_1) dá $\mathrm{pos}(x_k) < \mathrm{pos}(x_1)$. Juntos, $\mathrm{pos}(x_1) < \mathrm{pos}(x_k) < \mathrm{pos}(x_1)$, o que é absurdo.

($\Leftarrow$) Por indução em $n = |V(G)|$. Se $n = 1$ não há arcos, pois não há laços, e a única ordenação do vértice é uma ordenação topológica. Seja $n \geq 2$ e suponha o resultado válido para GDAs com $n-1$ vértices. Pela Proposição 5.2, G tem uma fonte s. O grafo $G' \coloneqq G[V(G) \setminus \{s\}]$ tem $n-1$ vértices e é acíclico, pois todo circuito orientado de G' é um circuito orientado de G. Pela hipótese de indução, G' tem uma ordenação topológica $w_1, \dots, w_{n-1}$. Afirmamos que $s, w_1, \dots, w_{n-1}$ é uma ordenação topológica de G. De fato, os arcos de G que não incidem em s são arcos de G' e respeitam a ordem de $w_1, \dots, w_{n-1}$. Como $d^-(s) = 0$, todo arco que incide em s é da forma (s, w_j), e s vem antes de w_j. □

A demonstração sugere um algoritmo: enquanto restarem vértices, remover uma fonte do grafo restante e colocá-la na próxima posição. Se em algum momento o grafo restante, que é não-vazio, não tem fonte, então, pela Proposição 5.2, ele não é acíclico, e portanto contém um circuito dirigido, que também é um circuito de G (o restante é um subgrafo induzido de G). Assim o algoritmo decide se G é acíclico e, em caso afirmativo, devolve uma ordenação topológica. O Exercício 85 pede para implementá-lo em tempo $O(|V| + |E|)$. Vejamos outro

algoritmo, baseado na busca em profundidade rotulada, que também decide se G é acíclico.

5.2.1 Ordenação topológica por busca em profundidade

A ideia é a seguinte. Se `sai`$[v]$ < `sai`$[u]$ para todo arco (u, v) do grafo, então listar os vértices em ordem decrescente de `sai`[ ] (cujos valores são distintos) dá uma ordenação topológica. Uma busca em profundidade rotulada garante essa desigualdade para todos os arcos que não são de retorno ascendente (isto é, de árvore, de retorno descendente e transversais; Proposição 5.1), e G é acíclico se e somente se a busca não produz arcos de retorno ascendente (Lema 5.1).

Lema 5.1

Após uma busca em profundidade rotulada completa em um grafo dirigido simples G, G contém um circuito orientado se e somente se existe um arco de retorno ascendente.

Demonstração. ($\Leftarrow$) Se (u, v) é de retorno ascendente, v é ancestral próprio de u, e o caminho da árvore de v a u junto com (u, v) é um circuito orientado. ($\Rightarrow$) Seja C um circuito orientado e x o vértice de C com menor `chega`. Quando x é descoberto, os outros vértices de C são brancos e há um caminho de x até cada um deles por vértices brancos; pelo teorema do caminho branco, todos são descendentes de x. O arco (y, x) de C que chega em x tem y descendente próprio de x, logo é de retorno ascendente. □

Para obter a lista sem ordenar, inserimos cada vértice no início de uma lista `ord` no instante em que ele termina. Como o último vértice a terminar é inserido por último, ele fica na frente: ao final, `ord` lista os vértices em ordem decrescente de `sai`[ ]. As variáveis cont, `chega`, `sai`, `p`, `ord` e `circ` são globais; o vetor `p` não é usado pelo algoritmo, apenas pela análise.

Algoritmo 26: VisitaOT(G, w).

Dado : um grafo dirigido G e um vértice w com `chega`$[w] = 0$.

Devolve : visita w e os vértices ainda não visitados alcançáveis a partir de w, atualizando `chega`, `sai`, `p`, `ord` e `circ`.

```
1  cont ← cont + 1;
2  chega[w] ← cont;
3  para cada u ∈ N⁺(w) faça
4      se chega[u] = 0 então
5          p[u] ← w;
6          VisitaOT(G, u);
7      senão
8          se sai[u] = 0 então circ ← verdadeiro;
9  cont ← cont + 1;
10 sai[w] ← cont;
11 insira w no início da lista ord;
```

Algoritmo 27: OrdTop(G).

Dado : um grafo dirigido G.

Devolve : uma ordenação topológica de G, se G é acíclico; caso contrário, a informação de que G contém circuito orientado.

```
1  para cada v ∈ V(G) faça
2      chega[v] ← 0;
3      sai[v] ← 0;
4      p[v] ← nil;
5  cont ← 0;
6  ord ← lista vazia;
7  circ ← falso;
8  para cada v ∈ V(G) faça
9      se chega[v] = 0 então VisitaOT(G, v);
10 se circ então
11     retorne G contém circuito orientado
12 retorne ord
```

Exemplo 5.2

Executando OrdTop(G) no grafo do Exemplo 5.1, com os vértices percorridos em ordem alfabética (em $V(G)$ e em cada $N^+(v)$), obtemos

v	a	b	c	d	e
chega[v]	1	2	6	3	7
sai[v]	10	5	9	4	8

e a lista ord no final é a, c, e, b, d, que lista os vértices em ordem decrescente de sai[ ] e é uma das ordenações do Exemplo 5.1. O arco (c, d) é transversal: é examinado quando d já terminou e, nesse caso, sai[d] < sai[c].

Análise do algoritmo.

Fixe um grafo dirigido G e considere uma execução de OrdTop(G). Pela estrutura recursiva do algoritmo valem as seguintes observações, que usaremos sem mais comentários.

(a) Para todo $v \in V(G)$ a chamada VisitaOT(G, v) é feita exatamente uma vez. De fato, ela só é feita se chega[v] = 0 e seus dois primeiros comandos tornam chega[v] ≠ 0; além disso, ao final do laço principal todo vértice tem chega ≠ 0. Em particular, cada arco (w, u) é examinado exatamente uma vez, no laço da chamada de w.

(b) O contador cont aumenta de uma unidade antes de cada atribuição a chega[ ] ou a sai[ ]. Logo esses $2|V(G)|$ valores são inteiros distintos e chega[v] < sai[v] para todo v.

(c) As chamadas recursivas são aninhadas: a chamada de u feita na chamada de w termina antes de w prosseguir. Em cada instante, os vértices com chega ≠ 0 e sai = 0 (chamadas *ativas*) são o vértice z cuja chamada está em execução e seus ancestrais na floresta definida por p[ ].

(d) Se chega[w] < chega[x] < sai[w], então x foi descoberto durante a chamada de w, portanto por uma chamada aninhada nela; logo x é descendente próprio de w e sai[x] < sai[w].

(As relações entre os intervalos [chega, sai] e ancestralidade são as da Proposição 3.2 e do Exercício 58.)

Lema 5.2: Exame de um arco

Considere o instante em que um arco $(w, u) \in E(G)$ é examinado no laço da linha 3, durante a chamada de w. Então vale exatamente uma das situações:

1. chega[u] = 0; então u é visitado em seguida, com p[u] = w, e sai[u] < sai[w];
2. chega[u] ≠ 0 e sai[u] ≠ 0; então sai[u] < sai[w];
3. chega[u] ≠ 0 e sai[u] = 0; então u é ancestral próprio de w (isto é, (w, u) é um arco de retorno ascendente) e sai[w] < sai[u].

Reciprocamente, se u é ancestral próprio de w, o arco (w, u)

cai na situação 3.

Demonstração. Os três casos esgotam as possibilidades para os valores de $\texttt{chega}[u]$ e $\texttt{sai}[u]$.

Caso 1. O algoritmo faz $\texttt{p}[u] \leftarrow w$ e chama VisitaOT(G, u), que termina, e atribui $\texttt{sai}[u]$, antes de w prosseguir. Como $\texttt{sai}[u] \neq 0$ implica $\texttt{chega}[u] \neq 0$ por (b), os três casos esgotam as possibilidades e são mutuamente exclusivos.

Caso 2. O valor $\texttt{sai}[u]$ já foi atribuído, enquanto $\texttt{sai}[w]$ ainda não, pois a chamada de w está em execução. Por (b), $\texttt{sai}[u] < \texttt{sai}[w]$.

Caso 3. A chamada de u está ativa. No instante do exame do arco, a chamada em execução é a de w, logo, por (c), os vértices ativos são w e seus ancestrais. Como G não tem laços, $u \neq w$, portanto u é ancestral próprio de w. Sendo a chamada de w aninhada na de u, ela termina antes, isto é, $\texttt{sai}[w] < \texttt{sai}[u]$.

Reciprocamente, se u é ancestral próprio de w então, durante a chamada de w, a chamada de u está ativa por (c), o que é a situação 3. □

Proposição 5.3: Circuitos e retornos ascendentes

Em uma execução de OrdTop(G) ocorre algum arco de retorno ascendente se e somente se G contém circuito orientado.

Demonstração. ($\Rightarrow$) Seja (w, u) um arco de retorno ascendente, isto é, u é ancestral próprio de w. Então existe $t \geq 1$ com $\texttt{p}^{(t)}[w] = u$ e, definindo $y_j := \texttt{p}^{(t-j)}[w]$ para $0 \leq j \leq t$, temos $y_0 = u$, $y_t = w$ e $\texttt{p}[y_j] = y_{j-1}$ para $1 \leq j \leq t$. Como $\texttt{p}[y_j]$ só é atribuído ao examinar o arco $(\texttt{p}[y_j], y_j)$, temos $(y_{j-1}, y_j) \in E(G)$. Os vértices $y_0, \ldots, y_t$ são distintos, pois $\texttt{chega}[\texttt{p}[v]] < \texttt{chega}[v]$ para todo v (o pai é descoberto antes do filho) e, com o arco $(w, u) = (y_t, y_0)$, formam o circuito orientado $y_0, y_1, \ldots, y_t, y_0$.

($\Leftarrow$) Seja $C = x_1, \ldots, x_k, x_1$ um circuito orientado em G. Trocando a numeração por uma rotação cíclica, podemos supor que $\texttt{chega}[x_1]$ é o menor valor de $\texttt{chega}$ entre os vértices de C. Mostramos por indução em i, para $i = 2, \ldots, k$, que

$$\texttt{chega}[x_1] < \texttt{chega}[x_i] < \texttt{sai}[x_1].$$

Em particular, por (d), cada x_i é descendente próprio de x_1 e $\texttt{sai}[x_i] < \texttt{sai}[x_1]$.

Para $i = 2$, por (e) aplicado a (x_1, x_2) temos $\texttt{chega}[x_2] < \texttt{sai}[x_1]$, e $\texttt{chega}[x_2] > \texttt{chega}[x_1]$ pela escolha de x_1, pois $x_2 \neq x_1$. Isso mostra Q_2.

Suponha Q_i para algum $i < k$. Por (d), x_i é descendente próprio de

x_1 e $\texttt{sai}[x_i] < \texttt{sai}[x_1]$; por (e) aplicado a (x_i, x_{i+1}),

$$\texttt{chega}[x_{i+1}] < \texttt{sai}[x_i] < \texttt{sai}[x_1],$$

e $\texttt{chega}[x_{i+1}] > \texttt{chega}[x_1]$ pela escolha de x_1. Isso mostra Q_{i+1}.

Assim x_k é descendente próprio de x_1, isto é, x_1 é ancestral próprio de x_k, e $(x_k, x_1) \in E(G)$ é um arco de retorno ascendente. □

Teorema 5.2: Corretude

Ao final de OrdTop(G) o indicador `circ` é verdadeiro se e somente se G contém circuito orientado. Se G é acíclico, a lista `ord` é uma ordenação topológica de G.

Portanto OrdTop(G) devolve uma ordenação topológica se G é acíclico e informa que G contém circuito orientado caso contrário.

Demonstração. O indicador `circ` é atribuído na linha 8 precisamente quando um arco é examinado na situação 3 do Lema 5.2, isto é, quando ocorre um arco de retorno ascendente. Pela Proposição 5.3, isso acontece se e somente se G contém circuito orientado.

Suponha G acíclico. Pelo que acabamos de provar, nenhum arco é examinado na situação 3, portanto todo arco (w, u) cai na situação 1 ou 2 e satisfaz

$$\texttt{sai}[u] < \texttt{sai}[w].$$

Por (a), cada vértice é inserido em `ord` exatamente uma vez, na linha 11, logo após receber seu valor de `sai[ ]`. Como as inserções são no início da lista e os valores de `sai[ ]` são atribuídos em ordem crescente, `ord` contém os vértices em ordem decrescente de `sai[ ]`. Assim, para todo arco (w, u), o vértice w aparece antes de u em `ord`, e `ord` é uma ordenação topológica de G. □

Teorema 5.3: Complexidade

Se G é dado por listas de adjacência $N^+[\ ]$ e a inserção no início de `ord` custa $O(1)$, então OrdTop(G) tem complexidade de tempo $O(|V| + |E|)$.

Demonstração. A inicialização custa $O(|V|)$ e o laço principal faz $|V|$ testes de custo $O(1)$, fora as chamadas. Por (a), cada vértice w dispara uma única chamada de VisitaOT, que, descontadas as chamadas recursivas, executa $O(1)$ operações fora do laço da linha 3 (incluindo a inserção na linha 11) e $d^+(w)$ iterações do laço, cada uma de custo $O(1)$. O custo total é, portanto,

$$O(|V|) + \sum_{w \in V} O\big(1 + d^+(w)\big) = O(|V| + |E|),$$

pois $\sum_{w \in V} d^+(w) = |E|$. □

Se G é dado por sua matriz de adjacência, percorrer $N^+(w)$ custa $\Theta(|V|)$ para cada w e o algoritmo tem complexidade $\Theta(|V|^2)$. A pilha de recursão tem profundidade até $|V|$, e o espaço total é $O(|V|)$ além da representação de G.

Por fim, pelo Teorema 5.1, o problema da ordenação topológica tem solução se e somente se G é acíclico; o algoritmo OrdTop decide se G é acíclico e, em caso afirmativo, devolve uma ordenação topológica.

5.3 Componentes fortemente conexas

Um ***caminho orientado*** é o grafo dirigido P definido por uma sequência de vértices distintos $P = x_1, x_2, \ldots, x_k$ tal que $(x_i, x_{i+1}) \in E(P)$ para todo $i \in \{1, 2, \ldots, k-1\}$. Nesse caso, dizemos que P é um caminho orientado ***de*** x_1 ***para*** x_k e usamos a notação $(x_1 \to x_k)$***-caminho***.

Definição 5.4: componente fortemente conexa

Uma ***componente fortemente conexa*** num grafo dirigido G é um subconjunto de vértices $W \subseteq V(G)$ maximal com respeito a seguinte propriedade:

para quaisquer $u, v \in W$ existem
- um $(u \to v)$-caminho orientado e
- um $(v \to u)$-caminho orientado,

ambos contidos em $G[W]$.

O diagrama abaixo mostra um grafo dirigido, cada um dos quatro círculos identifica um componente fortemente conexo do grafo.

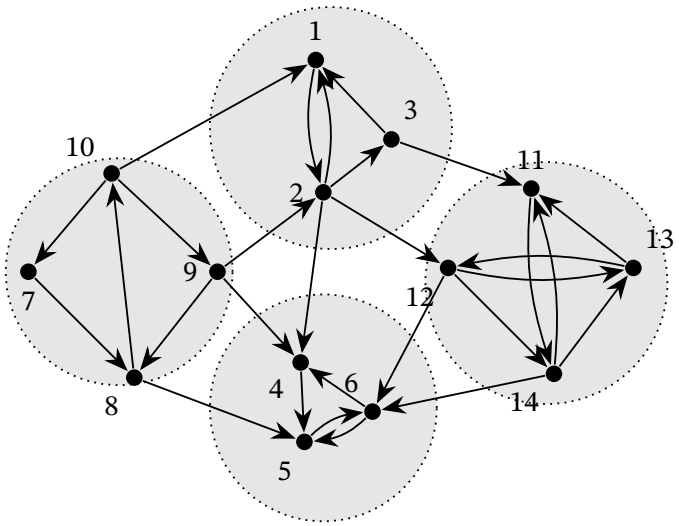


Figura 5.7: Um grafo dirigido e suas componentes fortemente conexas, destacadas em cinza.

Dado um grafo dirigido G, queremos computar os componentes fortemente conexos de G. Pra isso, definimos os seguintes dois grafos construídos a partir do grafo dirigido G

grafo transposto: denotado por $G^\intercal$ é o grafo dirigido obtido invertendo-se o sentido das arcos em $E(G)$: $G^\intercal = \big(V(G), \{(u, v) \in$

$V \times V : (v,u) \in E(G)\}$).

grafo das componentes: denotado por G^* é o grafo orientado com um vértice para cada componente fortemente conexo de G e (c_1, c_2) é uma arco em G^* se, e somente se, existe uma arco que sai de um vértice de c_1 e chega num vértice de c_2 (veja figura 5.8(a) abaixo).

Observação 5.1

Os grafos G e $G^\intercal$ têm os mesmos componentes fortemente conexos. De fato, se u e v estão no mesmo componente W de G então o $(u \to v)$-caminho em G com todos os vértices em W é um $(v \to u)$-caminho em $G^\intercal$ com todos os vértices em W e, analogamente, o $(v \to u)$-caminho em G é um $(v \to u)$--caminho em $G^\intercal$, portanto, u e v estão no componente W de $G^\intercal$.

Observação 5.2

O grafo G^* dos componentes fortemente conexos de G não contém circuito orientado. De fato, dados $v, v' \in c_i$ e $u, u' \in c_j$, para $i \neq j$, se existe $(u \to v)$-caminho em G então não pode existir $(v' \to u')$-caminho em G, caso contrário esses vértices estariam no mesmo componente fortemente conexo. Com isso, temos que G^* admite ordenação topológica.

Na figura abaixo mostramos (a) o grafo dos componentes fortemente conexos do grafo dirigido mostrado na Figura 5.7 e também (b) uma ordenação topológica dos vértices do grafo das componentes.

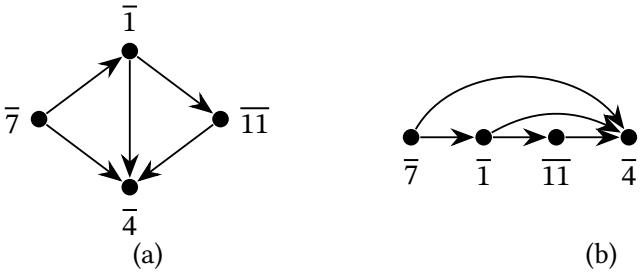


Figura 5.8: O grafo das componentes conexas (a) e uma ordenação topológica (b).

Agora, vamos descrever um algoritmo que, dado um grafo dirigido G, determina os componentes fortemente conexos de G. Em linhas gerais, a estratégia para computar os componentes fortemente conexos de um grafo dirigido G é

Algoritmo 28: CFC(G)

1 BP(G): busca em profundidade rotulada em G para determinar `sai`[v] para todo $v \in V(G)$;
2 Compute $G^\intercal$;
3 BP($G^\intercal$): busca em profundidade em $G^\intercal$ com o laço principal modificado para escolher os vértices em ordem decrescente dos valores de *sai*[ ] computado no passo 1;
4 Devolva cada árvore da segunda busca como um componente fortemente conexo de G.

No que segue sempre que nos referirmos aos valores de `chega`[ ] e `sai`[ ] trata-se do conteúdo desses vetores após a primeira busca em profundidade. Os seguintes resultados ajudam a entender o funcionamento do algoritmo. As demonstrações serão deixadas como exercício.

Lema 5.3

Sejam W e W' dois componentes fortemente conexos de G. Suponha que existe uma arco $(u, v) \in E(G)$ tal que $u \in W$ e $v \in W'$. Então

$$\max_{u \in W} \texttt{sai}(u) > \max_{v \in W'} \texttt{sai}(v).$$

Idéia da demonstração. Se a busca em profundidade rotulada chega no componente W antes de chegar no componente W', então todos os vértices de W' são descendentes do primeiro vértice visitado em W, o valor de `sai`[ ] desse vértice é maior que o valor de `sai`[ ] de todo vértice de W'. Se a busca em profundidade chega primeiro num vértice de W', então todos os vértices de W' serão visitados antes de qualquer visita a um vértice de W, portanto os valores de `sai`[ ] dos vértices em W são maiores que os valores de `sai`[ ] dos vértices de W'. □

Corolário 5.1

Sejam W e W' dois componentes fortemente conexos de G. Suponha que existe uma arco $(u, v) \in E(G^\intercal)$ tal que $u \in W$ e $v \in W'$. Então

$$\max_{u \in W} \texttt{sai}(u) < \max_{v \in W'} \texttt{sai}(v).$$

Demonstração. Imediato da definição de $G^\intercal$ e do resultado acima pois G e $G^\intercal$ têm os mesmos componentes conexos. □

A busca em profundidade do passo 3 começa no componente fortemente conexo W que tem o vértice $k \in V(G)$ com $\max_v$ `sai`(v). Essa

busca visita todos os vértices de W. Pelo corolário 5.1 somente esses vértices serão visitados, pois não há arcos de W para W' com $\max_{v\in W'} \texttt{sai}(v) < \max_{v\in W} \texttt{sai}(v)$, para qualquer outro componente fortemente conexo W' em G. Na segunda rodada do laço interno da busca em profundidade, o vértice u é tal que $\texttt{sai}(u) = \max\{\texttt{sai}(v) : v \in V(G) - W\}$ e, novamente pelo corolário 5.1, os vértices das componentes W'' tais que $\max_{v\in W''} \texttt{sai}(v) < \texttt{sai}(u)$ não são visitados; os únicos vértices fora do componente fortemente conexo de u que poderiam ser visitados são os da componente W, mas que já estão *visitados*; e assim por diante.

Análise e correção do algoritmo CFC.

Para facilitar a prova de que o algoritmo funciona corretamente vamos, primeiro, dar uma versão mais detalhada do algoritmo.

O algoritmo CFC(G) descrito a seguir faz uma busca em profundidade em G e depois uma busca em profundidade em $G^{\intercal}$ onde os vértices são visitados respeitando a ordenação descrita acima.

Em seguida mostramos que o algoritmo CFC( ) funciona corretamente. Lembramos que `sai[ ]` refere-se aos valores computados na primeira chamada da busca em profundidade.

O seguinte algoritmo usa o vetor `cfc[ ]` para identificar, no final da execução, os componentes fortemente conexos de G.

A busca em profundidade utilizada fica da seguinte forma

Algoritmo 29: BP(G, v, cc)

Dado : um grafo dirigido G.
Devolve : busca em profundidade rotulada modificada.

```
1 cfc(v) ← cc;
2 chega(v) ← cont;
3 cont ← cont + 1;
4 para cada u ∈ N⁻(v) faça
5     se chega(u) = 0 então BP(G, u, cc) ;
6 sai(v) ← cont;
7 cont ← cont + 1.
```

O parâmetro cc rotula os vértices de acordo com o componente fortemente conexo ao qual ele pertence, e o algoritmo CFC fica

Algoritmo 30: CFC(G)

Dado : um grafo dirigido G.

Devolve : componentes fortemente conexos.

```
1  para cada k ∈ V(G) faça
2      sai[k] ← 0;
3      chega[k] ← 0;
4  para cada k ∈ V(G) faça
5      se chega[k] = 0 então BP(G, k, 0) ;
6  L ← lista ordenada de V(G) por ordem decrescente
     de sai[ ];
7  compute G^T;
8  cont ← 0;
9  c ← 0;
10 para cada k ∈ V(G) faça
11     sai[k] ← 0;
12     chega[k] ← 0;
13 para cada k ∈ L faça
14     se chega[k] = 0 então
15         c ← c + 1;
16         BP(G^T, k, c);
```

Agora, damos uma prova de que o algoritmo CFC está correto.

Teorema 5.4

O algoritmo CFC(G) computa corretamente os componentes fortemente conexos do grafo dirigido G.

Demonstração. Vamos provar que, para todo $n \in \mathbb{N}$, após a n-ésima rodada do laço na linha 13 que: *Se $C = m$ então $W_i = \{v \; cfc(v) = i\}$ para $i \in \{1, 2, \dots, m\}\}$ são m componentes conexos de G.*

Para $n = 0$, antes da primeira rodada, a afirmação acima vale pois $\text{cfc}(v) = 0$ para todo $v \in V(G)$. Suponha que após a n-ésima rodada do laço na linha 13 temos $c = m$ e $W_i = \{v \; cfc(v) = i\}$ para $i \in \{1, 2, \dots, m\}\}$ são m componentes fortemente conexos de G. Consideremos a rodada $n+1$. Se o teste na linha 14 é falso então $c = m$ e após a rodada $n + 1$ temos os mesmos m componentes fortemente conexos de G. Vamos supor que o teste foi positivo, dessa forma $c = m+1$ e temos que provar que $W_i = \{v \, cfc(v) = i\}$, $1 \leq i \leq m+1$, são $m + 1$ componentes fortemente conexos de G.

Após a atribuição $c = m + 1$ na linha 15, temos uma chamada da busca em profundidade $\text{BP}(G^\intercal, k, m + 1)$, que faz $\text{cfc}[k] = m + 1$ e os vértices não-visitados, alcançáveis a partir de k, serão descendentes de k no final da busca e terão $\text{cfc}[\,] = m + 1$ na medida em que forem visitados. Seja W_{m+1} o componente fortemente conexo que contém k;

como $k \in L$ vale que

$$\text{sai}[k] = \max_{v \in W_{m+1}} \text{sai}(v) > \max_{u \in V \setminus \bigcup_{i=1}^{m+1} W_i} \text{sai}(u),$$

portanto, pelo corolário 5.1, não há arco $(u, v) \in W_{m+1} \times \left(V \setminus \bigcup_{i=1}^{m+1} W_i\right)$ em $G^\intercal$. Como os vértices de $\bigcup_{i=1}^{m} W_i$ já foram visitados (o teste da linha 14 falha) os valores de $cfc[\ ]$ desses vértices não são alterados por $\text{BP}(G^\intercal, k, m+1)$. Assim, nenhum vértice fora de W_{m+1} será descendente de k no final de $\text{BP}(G^\intercal, k, m+1)$. □

Observemos que se G é dado pela sua matriz de adjacências A, então a matriz transposta $A^\intercal$ representa o grafo transposto $G^\intercal$, dessa forma obtemos uma representação implícita de $G^\intercal$ em tempo constante no sentido de que $A^\intercal(i, j) = A(j, i)$. Se G é dado por uma lista de adjacências então a lista de adjacências de $G^\intercal$ pode ser computada em tempo $O(|V| + |E|)$ (verifique).

A linha 2 tem custo $O(|V|)$, o laço na linha 4 tem custo total $O(|V| + |E|)$, a ordenação custa $O(|V| \log |V|)$, o grafo $G^\intercal$ pode ser computado em tempo $O(|V| + |E|)$, a linha 8 tem custo $O(1)$, a 10 $O(|V|)$ e o laço na linha 13 tem custo $O(|V| + |E|)$.

Teorema 5.5

CFC(G) tem complexidade $O(|V| \log |V| + |E|)$.

Exercícios

Exercício 84 (***Fecho transitivo***). Se $D = (V, E)$ é um grafo dirigido então o fecho transitivo de D é o grafo $D^* = (V, E^*)$ onde

$$E^* = \{(i, j) \in V \times V : \text{ existe um } (i \to j)\text{-caminho em } D\}.$$

Escreva um algoritmo de tempo $O(|V|^3)$ para determinar o fecho transitivo de um grafo dirigido.

Exercício 85. Escreva um algoritmo que determina uma ordenação topológica removendo repetidamente uma fonte, como na demonstração do Teorema 5.1. Mantenha um vetor com os graus de entrada $d^-[\ ]$ e uma fila com as fontes atuais para obter tempo $O(|V| + |E|)$. Como decidir, ao fim, se G é acíclico?

Exercício 86. Quantas ordenações topológicas tem o grafo do Exemplo 5.1?

Exercício 87. Prove que um grafo dirigido acíclico com n vértices tem no máximo $\binom{n}{2}$ arcos e que existe um GDA com exatamente $\binom{n}{2}$ arcos.

Exercício 88. Prove que um grafo dirigido acíclico tem uma única ordenação topológica se e somente se contém um caminho orientado que passa por todos os vértices.

Exercício 89. Modifique o algoritmo OrdTop para que, quando G contém circuito orientado, ele devolva um circuito orientado de G. (Dica: use o vetor `p[ ]` e a demonstração da Proposição 5.3.)

Exercício 90. Mostre que listar os vértices em ordem ***crescente*** de `chega[ ]` não dá, em geral, uma ordenação topológica de um GDA. (Dica: considere o grafo com $V = \{1, 2, 3\}$ e arcos $(1, 3)$ e $(2, 3)$.)

Exercício 91. Mostre que um grafo dirigido é acíclico se e somente se todas as suas componentes fortemente conexas (Seção 5.3) têm um único vértice.

Exercício 92. Demonstre o lema 5.3.

Exercício 93. Mostre que o algoritmo 28 pode ser implementado com complexidade $O(|V| + |E|)$.

Exercício 94. Se ao invés de usarmos uma busca em profundidade em $G^\intercal$ por ordem decrescente de *sai*[ ] no algoritmo 30 usarmos uma busca em profundidade em G por ordem crescente de *chega*[ ] o algoritmo determinaria os componentes fortemente conexos do grafo?

Exercício 95. Se ao invés de usarmos uma busca em profundidade em $G^\intercal$ por ordem decrescente de *sai*[ ] no algoritmo 30 usarmos uma busca em profundidade em G por ordem crescente de *sai*[ ] o algoritmo determinaria os componentes fortemente conexos do grafo?

Exercício 96. Seja G um grafo dirigido e u e w vértices de G.

(a) Escrevemos $u \rightsquigarrow w$ se existe $(u \to w)$-caminho em G. Prove que $\rightsquigarrow$ é uma relação simétrica e transitiva em $V(G)$.

(b) Escrevemos $u \leftrightsquigarrow w$ se existem $(u \to w)$- e $(w \to u)$-caminhos em G. Prove que $\leftrightsquigarrow$ é uma relação de equivalência em $V(G)$. Nesse caso, quem são as classes de equivalência?

Exercício 97. Escreva um algoritmo para determinar se um grafo dirigido G é ***semiconexo***: para $u, v \in V(G)$ ou $u \rightsquigarrow v$ ou $v \rightsquigarrow u$. Determine a complexidade do algoritmo.

Exercício 98. Escreva um algoritmo de complexidade $O(|V|+|E|)$ para computar o grafo G^* dos componentes fortemente conexos de G.

Exercício 99. Escreva um algoritmo de complexidade $O(|V|+|E|)$ para determinar o grafo transposto de um grafo dado por uma lista de adjacências.

Índice

$d^+(v)$, 66
(V, E, ρ), 8
$(x_1 \to x_k)$-caminho, 77
A_G, 18
C^k, 5
$E(G)$, 1
$G + xy$, 40
$G - e$, 39
$G[M]$, 3
$G[U]$, 3
$G \simeq H$, 14
$G^{\complement}$, 9
$H \subseteq G$, 2
K^n, 10
K^n, 15
$N^+(v)$, $N^-(v)$, 66
$N_G(v)$, 2
P^k, 4
$V(G)$, 1
$\mathrm{d}_G(v)$, 2
$\mathrm{dist}_G(u, v)$, 6
$\mathrm{tr}(A)$, 26
$\Delta(G)$, 10
$\alpha(G)$, 11
$\chi(G)$, 10
$\delta(G)$, 5
$\omega(G)$, 11
$\nabla_G(L)$, 37
$\mathrm{d}(G)$, 10
SG, 66
$d^-(v)$, 66
k-cubo, 12
$p^{(t)}$, 36
$G \coloneqq (V, E)$, 1
Grafo de Petersen, 15

acíclico, 37, 39
adjacentes, 1
alcança, 7
ancestral próprio, 67
arco, 66
arcos, 8
aresta-transitivo, 25
arestas, 1
arestas adjacentes, 1
assintoticamente **muito** menor, 21
assintoticamente menor, 21
automorfismo, 25

caminho, 4
caminho orientado, 69, 77
chega em u, 66
cintura, 13
circuito, 4
circuitos fundamentais de G com respeito a T, 43
circuito orientado, 69
clique, 11
complemento, 9
completo, 10
complexidade de tempo polinomial, 20
componentes conexas, 7
componente fortemente conexa, 77
comprimento, 5, 6
concatenação, 12
conexo, 7
conjunto estável, 11
conjunto independente, 11
corte de arestas, 37
cortes fundamentais de G com respeito a T, 43
custo, 45

descendente próprio, 67
diagrama, 1
distância, 6
diâmetro, 13

espaço de cortes, 44
extremos, 1, 4

Fecho transitivo, 82
filho, 67
floresta, 39
folha, 39
fonte, 70

G - v, 39

G∩H, 10
G∪H, 9
grafo, 1
grafo com pesos nas arestas, 8
grafo de Fibonacci, 13
grafo dirigido, 66
grafo dirigido ou digrafo, 8
grafo dirigido simples, 66
grafo orientado, 8
grafo subjacente, 8, 66
grafo dirigido acíclico, 69
grau de v, 2
grau de entrada, 66
grau de saída, 66
grau máximo, 10
grau médio, 10
grau mínimo, 5

incide, 2
intersecção, 10
isomorfismo, 14
isomorfos, 14

lista de adjacência dirigida, 67

multigrafo, 8

operação elementar, 43
ordenação topológica, 70

pai, 67
passeio, 3
passeio fechado, 4
ponte, 43
problema da ordenação topológica, 70

regular, 11

sai de v, 66
subgrafo, 2
subgrafo gerador, 3
subgrafo induzido por M, 3
subgrafo induzido por U, 3
sumidouro, 70

T + uv - a, 43
Termo, 8
traço, 26

união, 9

vizinhança, 2
vértice-transitivo, 25
vértices, 1
vértices internos, 4

árvore, 39
árvore geradora, 41
árvore geradora mínima, 45

Bibliografia

BONDY, J. A.; MURTY, U. S. R. **Graph Theory**. New York: Springer, 2008. (Graduate Texts in Mathematics, v. 244).

BRASSARD, Gilles; BRATLEY, Paul. **Fundamentals of Algorithmics**. Englewood Cliffs: Prentice Hall, 1996.

BRUNO, J.; WEINBERG, L. A constructive graph-theoretic solution of the Shannon switching game. **IEEE Transactions on Circuit Theory**, v. 17, n. 1, p. 74–80, 1970.

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. **Algoritmos**: teoria e prática. 3. ed. Rio de Janeiro: Elsevier, 2012.

DASGUPTA, Sanjoy; PAPADIMITRIOU, Christos H.; VAZIRANI, Umesh V. **Algorithms**. Boston: McGraw-Hill, 2008.

DIESTEL, Reinhard. **Graph Theory**. 6. ed. Heidelberg: Springer, 2025. (Graduate Texts in Mathematics, v. 173).

ERICKSON, Jeff. **Algorithms**. 1. ed. [S. l.: s. n.], 2019. Capítulos 5–11. Disponível em: https://jeffe.cs.illinois.edu/teaching/algorithms/. Acesso em: 01/02/2026.

JOYNER, David; NGUYEN, Minh Van; COHEN, Nathann. **Algorithmic Graph Theory**. Version 0.7. [S. l.: s. n.], 2013. Disponível em: https://static.latexstudio.net/wp-content/uploads/2013/03/book.pdf. Acesso em: 01/02/2026.

LEHMAN, A. A solution of the Shannon switching game. **Journal of the Society for Industrial and Applied Mathematics**, v. 12, n. 4, p. 687–725, 1964.

MANBER, Udi. **Introduction to Algorithms**: a creative approach. [S. l.]: Addison-Wesley, 1998.

ROUGHGARDEN, Tim. **Algorithms Illuminated**. Part 2: Graph Algorithms and Data Structures. New York: Soundlikeyourself Publishing, 2018. Disponível em: https://www.rexresearch1.com/AlgorithmLibrary/AlgorithmsIlluminated2Roughgarden.pdf. Acesso em: 01/02/2026.

SEDGEWICK, Robert. **Algorithms in C**. Part 5: Graph Algorithms. 3. ed. Reading: Addison-Wesley, 2002. 482 p.